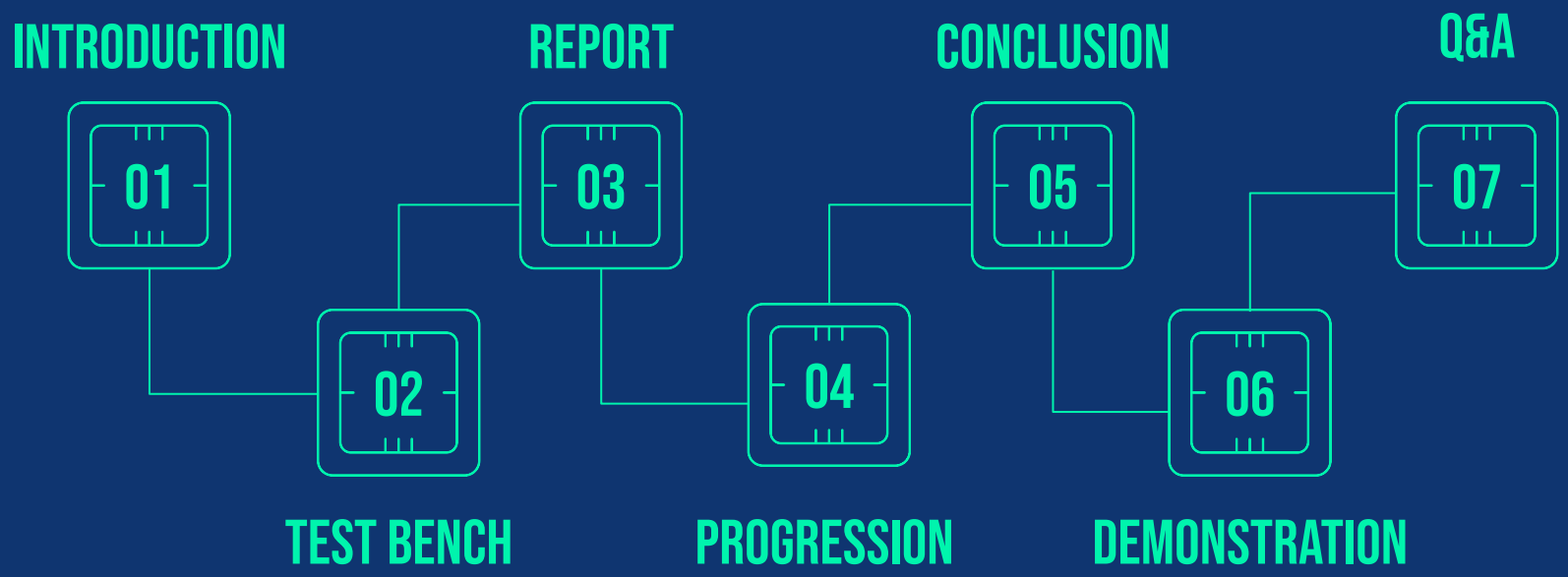
NIGHTHAWKS PROJECT PRESENTATION

Performance Profiling for C++ Serialization Libraries

Presentor: Evan Marino

PRESENTATION OUTLINE



TEAM INTRODUCTION

Team:

- Evan Marino [Product Owner]
- Kareena Khan [Scrum Master]
- Jacob Flores [Developer]
- Colin Harrell [Developer]
- Aaron Boylan [Developer]

Client:

- ASRC
- Daniel Dusharm

ASRC FEDERAL

Performance Profiling for C++ Serialization Libraries

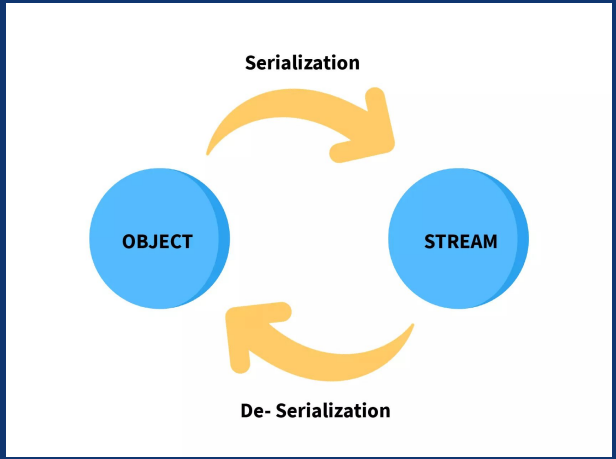
Project Overview

What are the project's objectives?

- Identify data serialization options for C++ code in a Linux environment
- Understand and compare different serialization methods (raw, binary, textual, etc.)
- Generate a comprehensive report on the results, including:
  - A comparison of the libraries against each other
  - Recommendations for specific use cases based on the findings

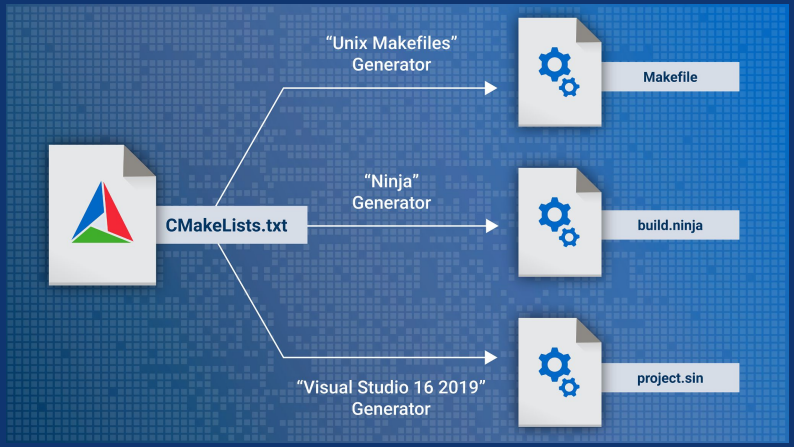
PROJECT INTRODUCTION

- Serialization - The process of converting data in code into a stream of bytes to save the state of an object in easily transmittable code.
- Working Environment - Ubuntu Linux operating on Oracle VirtualBox
- Version Control - GitHub
- Team Communication - Discord
- File Storage - Google Drive
- Project Management - Trello



TEST BENCH

- Created using CMake to build and link all of the necessary libraries and header files
- Create an instance of each Serialization library
  - Boost.Serialization
  - Cereal
  - Flatbuff
  - Protobuf




```
cmake_minimum_required(VERSION 3.12)
project(TestBench VERSION 1.0.0)

add_subdirectory(libs)
add_subdirectory(inputData)

include_directories(${CMAKE_SOURCE_DIR}/environmentSetup/bins_and_dirs)

add_executable(testBench testBench.cpp)
add_dependencies(testBench boostLib)
add_dependencies(testBench cerealLib)
add_dependencies(testBench flatbuffLib)
add_dependencies(testBench protobufLib)

target_link_libraries(testBench PUBLIC boostLib)
target_link_libraries(testBench PUBLIC cerealLib)
target_link_libraries(testBench PUBLIC flatbuffLib)
target_link_libraries(testBench PUBLIC protobufLib)
```

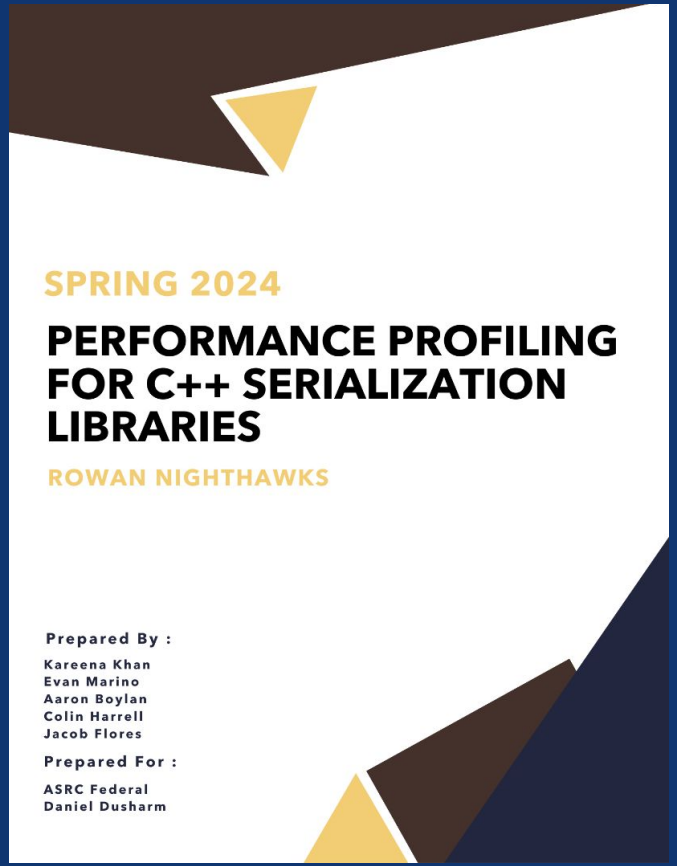
TEST BENCH CONT.

- Create a Test object
  - 1 Array of 5 Integers
  - 3 Integers
  - 3 Floats
  - 4 C++ standard strings
- Serialize and Deserialize this Test Object using each library x times. X is set by the user in the command line
- Track average time for these functions to take place, each on their own thread
- Find size of the final serialized file
- Write the output to the terminal

```
public:
    TestObject(){};

    TestObject(
        std::array<int, 5> integers,
        std::string string1,
        std::string string2,
        std::string string3,
        std::string string4,
        int int1,
        int int2,
        int int3,
        float float1,
        float float2,
        float float3
    ){
```

REPORT


SPRING 2024

PERFORMANCE PROFILING FOR C++ SERIALIZATION LIBRARIES

ROWAN NIGHTHAWKS

Prepared By :

Kareena Khan
Evan Marino
Aaron Boylan
Colin Harrell
Jacob Flores

Prepared For :

ASRC Federal
Daniel Dusharm

Table of Contents

1. *Abstract*
2. *TestBench Elements & Environment*
3. *Chosen Libraries & Previous Considerations*
4. *Observational and Researched Qualities Analysis*
   a. *Ease of Use*
   b. *Community Support*
   c. *Security Concerns*
5. *Metric Analysis Using the TestBench*
   a. *Speed*
   b. *Memory Usage*
   c. *Data Integrity*
6. *Overall Library Recommendation*

LIBRARY INTRODUCTIONS

- Boost.Serialization
  - Used for binary serialization, captures lots of metadata and is easy to adapt to any class
- Cereal
  - Used for binary, XML, and JSON serialization. Designed to be similar to Boost, but faster and more efficient, with its binary serialization capturing less metadata and taking up less space
- Protobuf
  - Google’s platform-independent and language-independent serialization library. Produces an XML-like file, but smaller and faster
- Flatbuffers
  - Originally created at Google for game development. Works similarly to Protobuf, but it is smaller and faster.

REPORT FINDINGS

EASE OF USE

- Easiest Library to work with:

  Cereal was easy to work with because it is header-only. This means that all we had to do was download the source code from their website, place the include folder in our git repository, and tell cmake where to find it. Cereal required a serialize method to be written for any class that it's going to serialize, but this was super easy to write. And, if Boost serialization has already been incorporated, no new serialize method needs to be implemented for Cereal.

- Hardest Library to work with:

  Protobuf was the hardest library to work with due to compatibility issues with their latest versions. After resolving those issues, Protobuf implementation was very similar to Flatbuff in the way that both included their own compilers for schema files and both required the use of buffers to serialize and deserialize data. The ultimate difference between the implementation of the two was that Protobuf's compiler generated an additional file, adding an extra step of linking it to the test bench through cmake.

COMMUNITY SUPPORT

- All Libraries had some sort of forum, typically a Google Groups or/and a StackOverflow you could ask questions in
- Most were unreliable in both forms
- The worst was Boost with only a Mailing List you had to subscribe to
- The Library with the best Community Support was FlatBuffers with a:
  - Responsive Discord
  - Google Groups
  - StackOverFlow
  - Gitter Chat Room
  - Additional Online Resources on their Overview Page

SECURITY HISTORY/CONCERNS

- Security Vulnerabilities were checked with CVE Details uses two scoring systems, EPSS and CVSS:
- **EPSS, or Exploit Prediction Scoring System**, is defined as: “a data-driven effort for estimating the likelihood (probability) that a software vulnerability will be exploited in the wild.”
- **CVSS, or Common Vulnerability Scoring System**, is defined as: “ a way to capture the principal characteristics of a vulnerability and produce a numerical score reflecting its severity. The numerical score can then be translated into a qualitative representation (such as low, medium, high, and critical) to help organizations properly assess and prioritize their vulnerability management processes.”
- For the Library Versions used in the Test Bench no serious concerns were reported

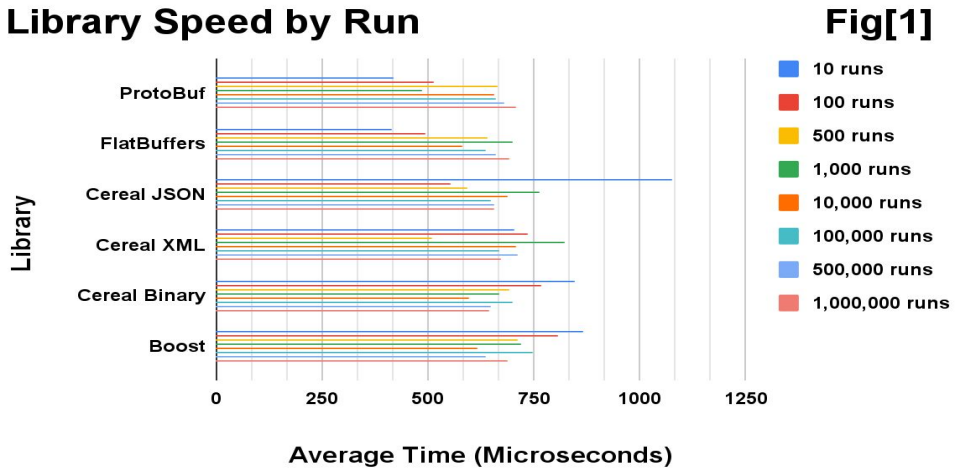
Library	Total Vulnerablities Reported	Highest CVSS and EPSS	Most Recent	Vulnerablities Reported for TestBench Version?
Boost Security	4	DATE: 1/17/2008 CVSS: 5.0 EPSS: 1.77% DETAILS: "The get_repeat_type function in basic_regex_creator.hpp in the Boost regex library (aka Boost.Regex) in Boost 1.33 and 1.34 allows context-dependent attackers to cause a denial of service (NULL dereference and crash) via an invalid regular expression"	DATE: 3/12/2013 CVSS: 5.0 EPSS: 0.76% DETAILS: " boost::locale::utf::utf_traits in the Boost.Locale library in Boost 1.48 through 1.52 does not properly detect certain invalid UTF-8 sequences, which might allow remote attackers to bypass input validation protection mechanisms via crafted trailing bytes."	VERSION: 1.82.0 NONE
Cereal Security	2	DATE: 3/30/2020 CVSS: 9.8 EPSS: 0.52% DETAILS: "An issue was discovered in USC iLab cereal through 1.3.0. It employs caching of std::shared_ptr values, using the raw pointer address as a unique identifier. This becomes problematic if an std::shared_ptr variable goes out of scope and is freed, and a new std::shared_ptr is allocated at the same address. Serialization fidelity thereby becomes dependent upon memory layout. In short, serialized std::shared_ptr variables cannot always be expected to serialize back into their original values. This can have any number of consequences, depending on the context within which this manifests."	SAME AS HIGHEST	VERSION: 1.3.2 NONE
Flatbuffers Security	2	DATE: 12/31/2020 CVSS: 9.8 EPSS: 0.22% DETAILS: "An issue was discovered in the flatbuffers crate before 0.6.1 for Rust. Arbitrary bytes can be reinterpreted as a bool, defeating soundness."	SAME AS HIGHEST	VERSION: 23.5.26 NONE
ProtoBuf Security	2	DATE: 9/25/2017 CVSS: 8.8 EPSS: 1.68% DETAILS: "protobuf allows remote authenticated attackers to cause a heap-based buffer overflow."	DATE: 1/26/2022 CVSS: 6.5 EPSS: 0.05% DETAILS: " Nullptr dereference when a null char is present in a proto symbol. The symbol is parsed incorrectly, leading to an unchecked call into the proto file's name during generation of the resulting error message. Since the symbol is incorrectly parsed, the file is nullptr. We recommend upgrading to version 3.15.0 or greater."	VERSION: 3.12 Potentially, 3.15 and above has been recommended

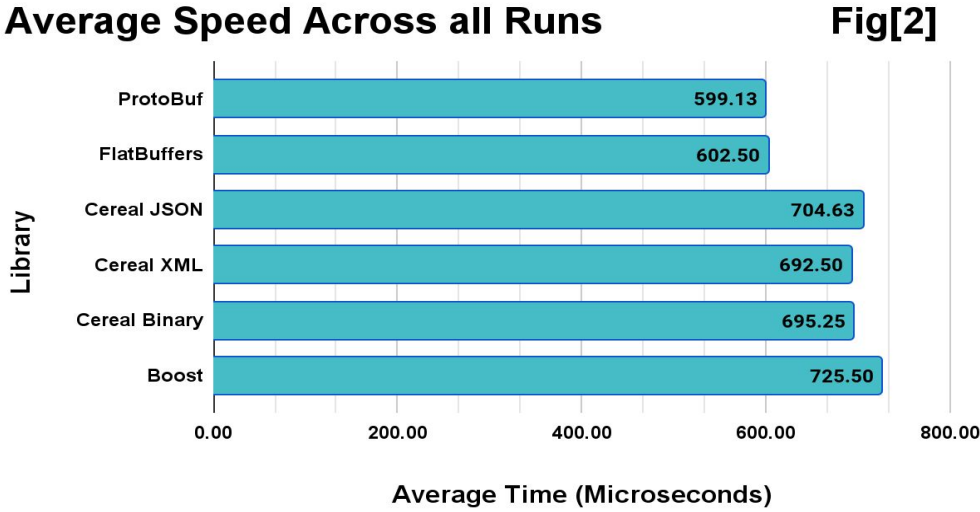
Quoted from FIRST.ORG

SPEED

Ranking:

1) ProtoBuf
2) FlatBuffers
3) Cereal XML
4) Cereal Binary
5) Cereal JSON
6) Boost

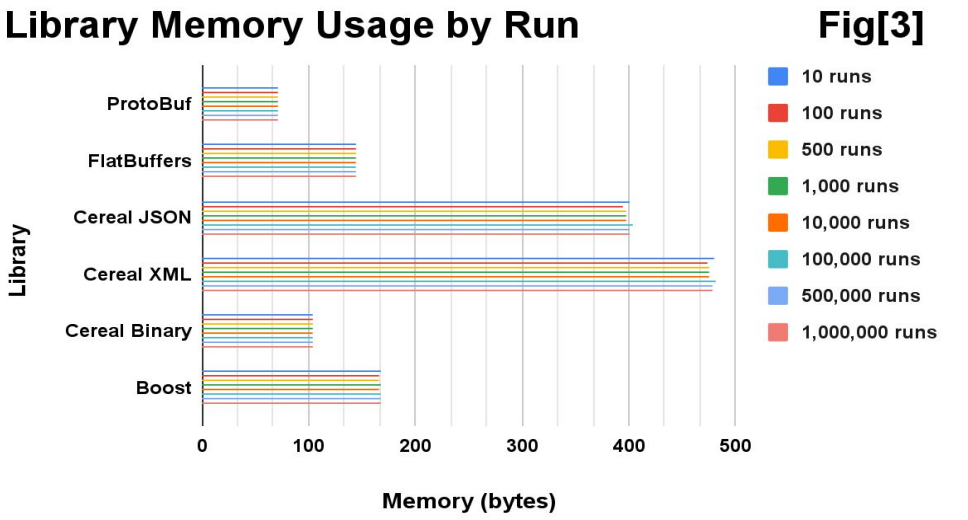


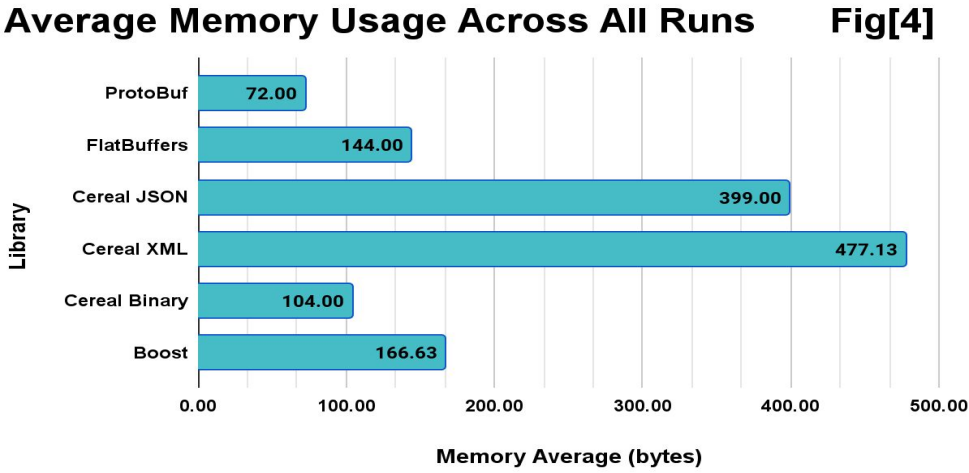


MEMORY USAGE

Ranking:

1) ProtoBuf
2) Cereal Binary
3) FlatBuffers
4) Boost
5) Cereal JSON
6) Cereal XML





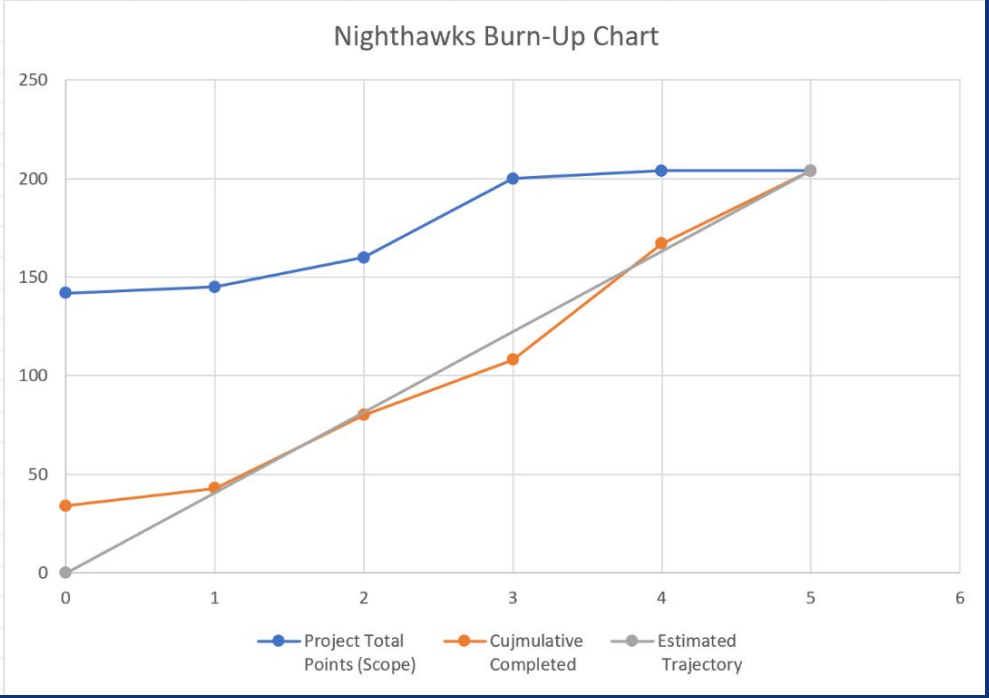
DATA INTEGRITY

- To test Data Integrity each pre-serialized and post-serialized file were tested to see if their contents matched after the serialization process.
- This was tested with varying quantities of Runs
- No matter the amount of runs or the differences between the libraries, each library was equally reliable and able to maintain the contents of the object it was serializing

Is the Object Correctly Deserialized?								
Library	10 runs	100 runs	500 runs	1,000 runs	10,000 runs	100,000 runs	500,000 runs	1,000,000 runs
ProtoBuf	yes	yes	yes	yes	yes	yes	yes	yes
FlatBuffers	yes	yes	yes	yes	yes	yes	yes	yes
Cereal JSON	yes	yes	yes	yes	yes	yes	yes	yes
Cereal XML	yes	yes	yes	yes	yes	yes	yes	yes
Cereal Binary	yes	yes	yes	yes	yes	yes	yes	yes
Boost	yes	yes	yes	yes	yes	yes	yes	yes

PROGRESSION

Sprint	Project Total Points (Scope)	Completed	Cujmulative Completed	Estimated Trajectory
0	142	34	34	0
1	145	9	43	#N/A
2	160	37	80	#N/A
3	200	28	108	#N/A
4	204	59	167	#N/A
5	204	37	204	204



PROGRESSION

Sprint 5 Done

- Reliability (report) — 1, 2/2, 5
- Work cited page — 5
- Create Graphs for Report — 3/3, 5
- Performance (report) — 1, 5/5, 5
- Recommendation (report) — 1, 1/2, 3
- Ease of Use (report) — 1, 2/3, 7

+ Add a card

Sprint 4 Done

- Validation Document — Apr 11, 5
- Explain Libraries (report) — 3/3, 5
- Incorporate each library into Testbench — 8/8, 17, JF
- Finalize Environment setup — 3
- Mock presentation — Apr 15, 5
- Write README in github — 1, 5
- Data Integrity Evaluation — 2/2, 7
- Develop Design for C++ test programs — 3/3, 7
- Change # of iterations to be chosen — 5

+ Add a card

Sprint 3 Done

- Design and Architecture — Apr 5, 5
- Community Support (report) — 1, 5/5, 3
- Title Page (report) — 1
- Start Abstract (report) — 3
- Table of Contents (report) — 1
- Memory Evaluation — 2/2, 5
- Speed Evaluation — 2/2, 5
- Introduce Testbench (report) — 3/3, 2
- Security Concerns (report) — 1, 4/4, 3

+ Add a card

Sprint 2 Done

- Complete Definition of Done Doc — Mar 8, 3
- Complete Requirements Document — Mar 1, 3
- Create basic library interface for Testbench — 2/2, 11, AB
- Finalize Research on Community Support — 3
- Create general input for each library — 3
- Report Outline — 7
- Begin Writing about Ease of Use/Security/Community Support — 7

+ Add a card

Sprint 1 Done

- Select 4 Open source serde libraries — 6/7, 7, JF, AB, CH
- Create abstract for Test Bench design — 6/6, 5
- Continue Researching Security Issues — 2
- Get ProtoBuf and FlatBuf downloaded and tested — 3/4, 7, CH, JF

+ Add a card

CONCLUSION

- Our product will provide valuable insight for ASRC in utilizing data serialization
- The hardest part of serialization that we faced was getting the libraries set up in our Linux VM. Figuring out which header files and binary files were necessary and how to link them was troublesome
- We have extracted valuable data about 4 different Serialization libraries running in a Linux environment
- We concluded that the best option comes down to whether Ease of Use or Performance is more important, as Protobuf has the best performance numbers, Cereal is by far the easiest to use, and Flatbuffers was the best middle ground and has the best community support

DEMONSTRATION

With Development Team